

Introducción a la Arquitectura y Organización de Computadores

Alejandro Furfaro

17 de agosto de 2023

Agenda

- 1 Introducción
 - Arquitectura y Organización
 - Instruction Set Architecture (ISA)
 - Organización y Hardware
- 2 Objetivos de estudiar organización y hardware
- 3 Orígenes
 - En solo 80 años
 - Modernidad
 - Pipeline
- 4 Nace un paradigma de Arquitectura: RISC
 - Antecedentes

1 Introducción

- Arquitectura y Organización
- Instruction Set Architecture (ISA)
- Organización y Hardware

2 Objetivos de estudiar organización y hardware

3 Orígenes

4 Nace un paradigma de Arquitectura: RISC

Arquitectura vs Microarquitectura

Arquitectura

Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia los modelos anteriores).

Arquitectura vs Microarquitectura

Arquitectura

Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia los modelos anteriores).

- Registros

Arquitectura vs Microarquitectura

Arquitectura

Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia los modelos anteriores).

- Registros
- Set de instrucciones

Arquitectura vs Microarquitectura

Arquitectura

Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia los modelos anteriores).

- Registros
- Set de instrucciones
- Estructuras de memoria (descriptores de segmento y de página p. ej.)

Arquitectura vs Microarquitectura

Arquitectura

Es el conjunto de recursos accesibles para el programador, que por lo general se mantienen a lo largo de los diferentes modelos de procesadores de esa arquitectura (puede evolucionar pero la tendencia es mantener compatibilidad hacia los modelos anteriores).

- Registros
- Set de instrucciones
- Estructuras de memoria (descriptores de segmento y de página p. ej.)

Micro Arquitectura

Es la implementación en el silicio de la arquitectura. Lo que está detrás del set de registros y del modelo de programación. Puede ser muy simple o sumamente robusta y poderosa. Cambia de un modelo a otro dentro de una misma familia.

Definición de la Arquitectura de un Computador

- Es sin dudas la tarea mas ardua de un diseñador.

Definición de la Arquitectura de un Computador

- Es sin dudas la tarea mas ardua de un diseñador.
- Se trata de definir los aspectos mas relevantes en la arquitectura de un computador que maximicen su rendimiento, sin dejar de satisfacer otras limitaciones impuestas por los usuarios, como un costo económico que lo haga accesible, o un consumo de energía moderado.

Definición de la Arquitectura de un Computador

- Es sin dudas la tarea mas ardua de un diseñador.
- Se trata de definir los aspectos mas relevantes en la arquitectura de un computador que maximicen su rendimiento, sin dejar de satisfacer otras limitaciones impuestas por los usuarios, como un costo económico que lo haga accesible, o un consumo de energía moderado.
- Comprende el diseño del set de instrucciones, el manejo de la memoria y sus modos de direccionamiento, los restantes bloques funcionales que componen el CPU, el diseño lógico, y el proceso de implementación

Definición de la Arquitectura de un Computador

- Es sin dudas la tarea mas ardua de un diseñador.
- Se trata de definir los aspectos mas relevantes en la arquitectura de un computador que maximicen su rendimiento, sin dejar de satisfacer otras limitaciones impuestas por los usuarios, como un costo económico que lo haga accesible, o un consumo de energía moderado.
- Comprende el diseño del set de instrucciones, el manejo de la memoria y sus modos de direccionamiento, los restantes bloques funcionales que componen el CPU, el diseño lógico, y el proceso de implementación
- La implementación comprende el diseño del circuito integrado, su encapsulado, montaje, alimentación y refrigeración.

Definición de la Arquitectura de un Computador

skills necesarios

No es posible realizar esta tarea con éxito sin tener manejar de manera solvente varias tecnologías diferentes:

Definición de la Arquitectura de un Computador

skills necesarios

No es posible realizar esta tarea con éxito sin tener manejar de manera solvente varias tecnologías diferentes:

- Diseño lógico.

Definición de la Arquitectura de un Computador

skills necesarios

No es posible realizar esta tarea con éxito sin tener manejar de manera solvente varias tecnologías diferentes:

- Diseño lógico.
- Tecnología de encapsulado

Definición de la Arquitectura de un Computador

skills necesarios

No es posible realizar esta tarea con éxito sin tener manejar de manera solvente varias tecnologías diferentes:

- Diseño lógico.
- Tecnología de encapsulado
- Funcionamiento y diseño de compiladores y de Sistemas Operativos.

1 Introducción

- Arquitectura y Organización
- **Instruction Set Architecture (ISA)**
- Organización y Hardware

2 Objetivos de estudiar organización y hardware

3 Orígenes

4 Nace un paradigma de Arquitectura: RISC

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Clases de ISA. ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Clases de ISA. ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Direccionamiento de Memoria. Alineación obligatoria de datos vs. administración de a bytes.

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Clases de ISA. ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Direccionamiento de Memoria. Alineación obligatoria de datos vs. administración de a bytes.

Modos de Direccionamiento. Como se especifican los operandos.

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Clases de ISA. ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Direccionamiento de Memoria. Alineación obligatoria de datos vs. administración de a bytes.

Modos de Direccionamiento. Como se especifican los operandos.

Tipos y tamaños de operandos. Enteros, Punto Flotante, Punto Fijo. Diferentes tamaños y precisiones.

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Clases de ISA. ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Direccionamiento de Memoria. Alineación obligatoria de datos vs. administración de a bytes.

Modos de Direccionamiento. Como se especifican los operandos.

Tipos y tamaños de operandos. Enteros, Punto Flotante, Punto Fijo. Diferentes tamaños y precisiones.

Operaciones. Pocas Simples (RISC) o Muchas Complejas (CISC).

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Clases de ISA. ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Direccionamiento de Memoria. Alineación obligatoria de datos vs. administración de a bytes.

Modos de Direccionamiento. Como se especifican los operandos.

Tipos y tamaños de operandos. Enteros, Punto Flotante, Punto Fijo. Diferentes tamaños y precisiones.

Operaciones. Pocas Simples (RISC) o Muchas Complejas (CISC).

Instrucciones de control de flujo Saltos condicionales, calls.

Definiendo un ISA

Nos referimos a *Instruction Set Architecture*, como el set de instrucciones visibles por el programador. Es además el límite entre el software y el hardware.

Clases de ISA. ISAs con Registros de Propósito general vs. Registros dedicados. ISAs registro-memoria vs. ISAs Load Store.

Direccionamiento de Memoria. Alineación obligatoria de datos vs. administración de a bytes.

Modos de Direccionamiento. Como se especifican los operandos.

Tipos y tamaños de operandos. Enteros, Punto Flotante, Punto Fijo. Diferentes tamaños y precisiones.

Operaciones. Pocas Simples (RISC) o Muchas Complejas (CISC).

Instrucciones de control de flujo Saltos condicionales, calls.

Longitud del código Instrucciones de tamaño fijo vs. variable.

1 Introducción

- Arquitectura y Organización
- Instruction Set Architecture (ISA)
- Organización y Hardware

2 Objetivos de estudiar organización y hardware

3 Orígenes

4 Nace un paradigma de Arquitectura: RISC

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y del set de instrucciones.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y del set de instrucciones.
- Implementación de paralelismo a nivel de Instrucciones, y/o de datos.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y del set de instrucciones.
- Implementación de paralelismo a nivel de Instrucciones, y/o de datos.
- Podemos así encontrar procesadores que poseen la misma ISA pero una organización muy diferente.

Microarquitectura = Organización + Hardware

Organización

Se refiere a los detalles de implementación de la ISA. Es decir:

- Organización e interconexión de la memoria.
- Diseño de los diferentes bloques de la CPU y del set de instrucciones.
- Implementación de paralelismo a nivel de Instrucciones, y/o de datos.
- Podemos así encontrar procesadores que poseen la misma ISA pero una organización muy diferente.

Ejemplo:

Los procesadores AMD FX y los Intel Core i7, tienen la misma ISA, sin embargo organizan su caché y su motor de ejecución de maneras diferentes.

Microarquitectura = Organización + Hardware

Hardware

- Se refiere a los detalles de diseño lógico y tecnología de fabricación.

Microarquitectura = Organización + Hardware

Hardware

- Se refiere a los detalles de diseño lógico y tecnología de fabricación.
- Existirán por lo tanto, procesadores con la misma ISA y la misma organización, pero absolutamente distintos a nivel de hardware y diseño lógico detallado.

Microarquitectura = Organización + Hardware

Hardware

- Se refiere a los detalles de diseño lógico y tecnología de fabricación.
- Existirán por lo tanto, procesadores con la misma ISA y la misma organización, pero absolutamente distintos a nivel de hardware y diseño lógico detallado.

Ejemplo:

El Pentium 4, diseñado para desktop, y el Pentium 4 M para notebooks. Este último tiene una cantidad de lógica para control del consumo de energía, siendo su hardware muy diferente del del Pentium 4 desktop.

¿Porque necesitamos saber todo esto?

¿Porque necesitamos saber todo esto?

- Para comprender lo que ocurre debajo del software.

¿Porque necesitamos saber todo esto?

- Para comprender lo que ocurre debajo del software.
- Comprender como los diseños de hardware impactan en el software y en el programador

¿Porque necesitamos saber todo esto?

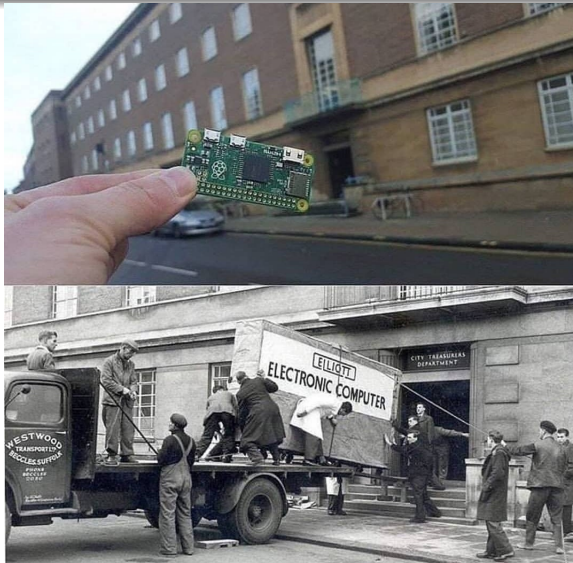
- Para comprender lo que ocurre debajo del software.
- Comprender como los diseños de hardware impactan en el software y en el programador
- Estar en condiciones de cruzar verticalmente las capas que separan el silicio de la aplicación.

¿Porque necesitamos saber todo esto?

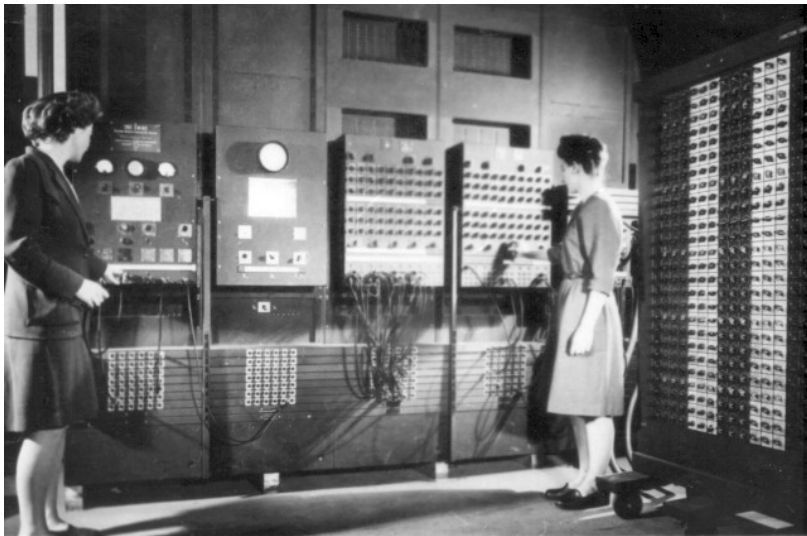
- Para comprender lo que ocurre debajo del software.
- Comprender como los diseños de hardware impactan en el software y en el programador
- Estar en condiciones de cruzar verticalmente las capas que separan el silicio de la aplicación.
- Comprender las nociones básicas que rigen el diseño de un sistema de cómputo moderno

- 1 Introducción
- 2 Objetivos de estudiar organización y hardware
- 3 Orígenes**
 - En solo 80 años
 - Modernidad
 - Pipeline
- 4 Nace un paradigma de Arquitectura: RISC

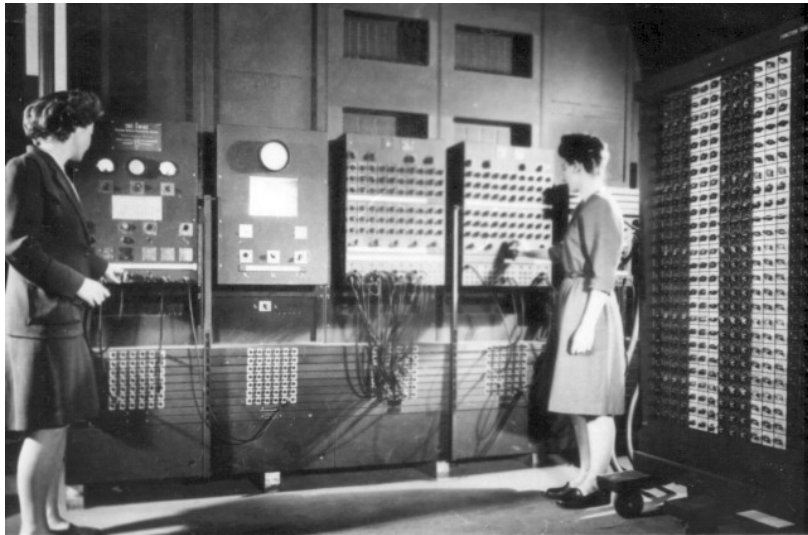
En solo algo menos de 80 años



ENIAC “primer” Computadora de Propósito general



ENIAC “primer” Computadora de Propósito general



Tal parece que la Z1 en Alemania fue algo anterior.

Pero esta está considerada la primera. No utilizaba programa almacenado, sino que se programaba mediante 6000 interruptores.

Consumo 160 KW,

Peso: 27 Toneladas,

Superficie: 167 m^2 .

ENIAC BackPlane. Observar las válvulas termoiónicas



Modelo de Von Mewmann

- 1945, “First Draft of a Report on the EDVAC. John Von Newmann”.

Modelo de Von Mewmann

- 1945, “First Draft of a Report on the EDVAC. John Von Newmann”.
- Este reporte borrador describe la organización y arquitectura de una máquina de cómputo binaria y electrónica en la que se detallan las características y recursos de arquitectura de los componentes principales.

Modelo de Von Mewmann

- 1945, “First Draft of a Report on the EDVAC. John Von Newmann”.
- Este reporte borrador describe la organización y arquitectura de una máquina de cómputo binaria y electrónica en la que se detallan las características y recursos de arquitectura de los componentes principales.
- Tal vez su mérito principal sea el de haber sido la primer publicación de la arquitectura y organización de un computador basado en los postulados de Turing de una década antes. Obviamente la **EDVAC** (por **E**lectronic **D**iscrete **V**ariable **A**utomatic **C**omputer) era una máquina Turing completa.

Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).

Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).
- De modo que sin perjuicio de los valiosos aportes de Von Newmann, estos conceptos salieron de otra mente brillante: la de Alan Turing.

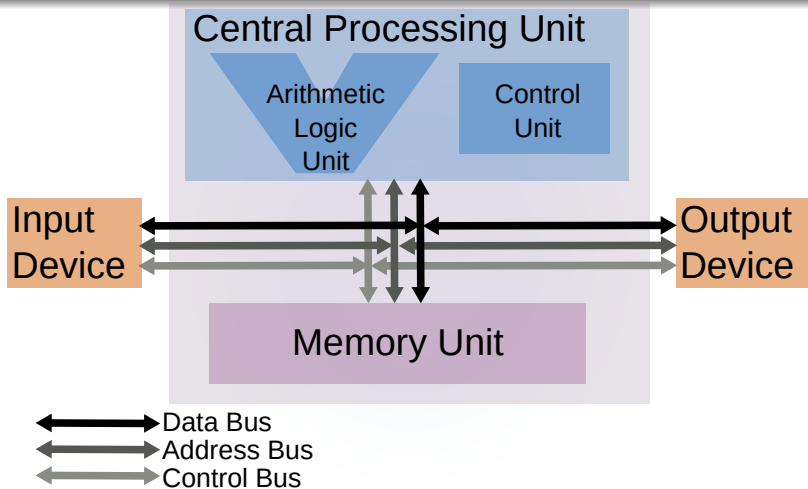
Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).
- De modo que sin perjuicio de los valiosos aportes de Von Newmann, estos conceptos salieron de otra mente brillante: la de Alan Turing.
- No obstante, los avances relativos de la EDVAC debidos a los trabajos de Von Newmann, y su publicación en el trabajo antes mencionado fue fundamental para la difusión del modelo de programa almacenado.

Modelo de Von Mewmann

- Inaugurando el lema “publish or perish”, esta arquitectura por ser la primera bien documentada, se convirtió en sinónimo de programa almacenado (recordemos que la máquina de Turing había postulado las bases teóricas de este concepto 9 años antes).
- De modo que sin perjuicio de los valiosos aportes de Von Newmann, estos conceptos salieron de otra mente brillante: la de Alan Turing.
- No obstante, los avances relativos de la EDVAC debidos a los trabajos de Von Newmann, y su publicación en el trabajo antes mencionado fue fundamental para la difusión del modelo de programa almacenado.
- Esta publicación fue muy influyente durante décadas en el diseño y concepción de computadores, y éste es su significativo aporte.

Modelo de Von Mewmann



Modelo simplificado de Von Newmann: Tres unidades unidas por tres buses.

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores

- **Central Arithmetic (CA):** Una Unidad de Procesamiento Aritmético Lógica y Registros.

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores

- **Central Arithmetic (CA):** Una Unidad de Procesamiento Aritmético Lógica y Registros.
- **Central Control (CC):** Implementación de la máquina de estados, con registro de instrucciones y registro Contador de Programa.

Modelo de Von Mewmann

Esta organización fue dominante durante la primer y segunda generación de Computadores

- **Central Arithmetic (CA):** Una Unidad de Procesamiento Aritmético Lógica y Registros.
- **Central Control (CC):** Implementación de la máquina de estados, con registro de instrucciones y registro Contador de Programa.
- **Memory (M):** Almacena el código y los Datos. Dos cuestiones: (1) Esta idea ya estaba en la **ENIAC** y en otros diseños anteriores, y (2) Con el tiempo se advirtió que este escenario limita la performance ya que no permite leer una instrucción y un dato a la vez, escenario conocido como “Von Newmann bottleneck”.

Modelo de Von Mewmann - Resumen

- **Entrada (I):** Se trata de mecanismos para que se puedan ingresar al computador comandos y datos desde el exterior.

Modelo de Von Mewmann - Resumen

- **Entrada (I):** Se trata de mecanismos para que se puedan ingresar al computador comandos y datos desde el exterior.
- **Salida (O):** Se trata de mecanismos para que el computador pueda enviar resultados hacia el mundo exterior.

Modelo de Von Mewmann - Resumen

- **Entrada (I):** Se trata de mecanismos para que se puedan ingresar al computador comandos y datos desde el exterior.
- **Salida (O):** Se trata de mecanismos para que el computador pueda enviar resultados hacia el mundo exterior.
- **Memoria Externa (R):** Se trata de un medio de almacenamiento de mayor capacidad que la Unidad de Memoria en donde solo están el código y los datos del programa en ejecución. En la Memoria externa podemos tener una colección de programas y datos para copiar de a una vez en la Unidad de Memoria y lanzar a ejecutar. Es lo que hoy conocemos como Storage. En ese entonces se trataba de Cintas perforadas por ejemplo.

Modelo de Von Meumann

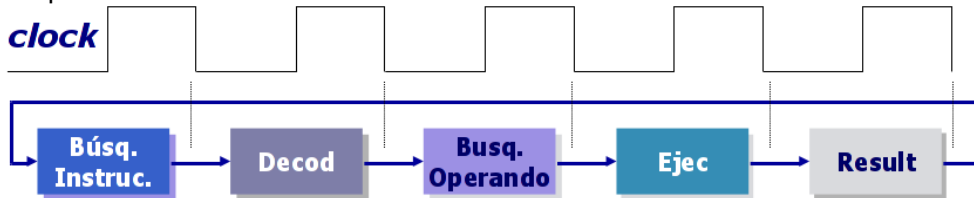
- 1 Modelo de programa almacenado.

Modelo de Von Meumann

- 1 Modelo de programa almacenado.
- 2 Datos y programa en el mismo (único) banco de memoria.

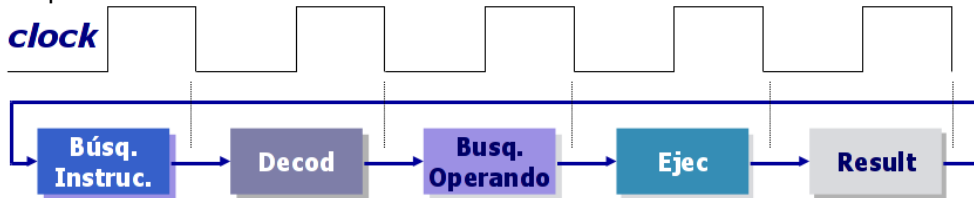
Modelo de Von Meumann

- 1 Modelo de programa almacenado.
- 2 Datos y programa en el mismo (único) banco de memoria.
- 3 Máquina de estados



Modelo de Von Meumann

- 1 Modelo de programa almacenado.
- 2 Datos y programa en el mismo (único) banco de memoria.
- 3 Máquina de estados



- 4 No está claro cual fue la primera implementación, pero Manchester Baby, IBM-SSEC, EDVAC, BINAC, entre otras fueron de las primeras (alrededor de 1948 todas ellas)

Modelo o Arquitectura Harvard

Modelo o Arquitectura Harvard

- En 1945, cuando Von Neumann publica el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas alguien mas iba a intentar leer desde memoria una operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).

Modelo o Arquitectura Harvard

- En 1945, cuando Von Neumann publica el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas alguien mas iba a intentar leer desde memoria una operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).
- El postulado de Von Neumann “una única Unidad de Memoria para datos y código”, definitivamente no encaja en el nuevo paradigma.

Modelo o Arquitectura Harvard

- En 1945, cuando Von Neumann publica el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas alguien mas iba a intentar leer desde memoria una operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).
- El postulado de Von Neumann “una única Unidad de Memoria para datos y código”, definitivamente no encaja en el nuevo paradigma.
- En este nuevo escenario, una memoria que almacene ambas entidades se transforma lisa y llanamente en un cuello de botella.

Modelo o Arquitectura Harvard

- En 1945, cuando Von Neumann publica el reporte de la EDVAC, tal vez era difícil imaginar que en solo cuestión de algo mas de dos décadas alguien mas iba a intentar leer desde memoria una operación y los datos necesarios para llevarla adelante **al mismo tiempo** (o al menos que todo se comporte como si así fuese).
- El postulado de Von Neumann “una única Unidad de Memoria para datos y código”, definitivamente no encaja en el nuevo paradigma.
- En este nuevo escenario, una memoria que almacene ambas entidades se transforma lisa y llanamente en un cuello de botella.
- Surge lo que se conoce como “Von Neumann bottleneck”.

Modelo o Arquitectura Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944. Es decir ni siquiera un computador de la primer generación, y obviamente anterior a la **EDVAC**.

Modelo o Arquitectura Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944. Es decir ni siquiera un computador de la primer generación, y obviamente anterior a la **EDVAC**.
- Mas aun: 760.000 engranajes, 800km de cables, basada en la maquina analítica de Babagge, y para colmo ¡¡¡trabajaba en Decimal!!!

Modelo o Arquitectura Harvard

- Recién por entonces y ante ese problema concreto, algunos diseñadores repararon en el diseño de la **Mark I**, un computador electromecánico desarrollado por IBM y la Universidad de Harvard entre 1939 y 1944. Es decir ni siquiera un computador de la primer generación, y obviamente anterior a la **EDVAC**.
- Mas aun: 760.000 engranajes, 800km de cables, basada en la maquina analítica de Babagge, y para colmo ¡¡¡trabajaba en Decimal!!!
- ¿Que podía tener de interesante esta pieza de museo que insumía 0.3 a 10 segundos por cálculo? (no obstante lo cual se utilizó hasta 1959...)

Modelo o Arquitectura Harvard

- Esta reliquia ni electrónica ni binaria tenía en sus entrañas la solución al “Von Newmann bottleneck”. Así son las cosas. . .

Modelo o Arquitectura Harvard

- Esta reliquia ni electrónica ni binaria tenía en sus entrañas la solución al “Von Newmann bottleneck”. Así son las cosas. . .
- Su código ingresaba mediante una cinta de papel perforada, y los operandos mediante contadores electromecánicos. Es decir, las instrucciones y los datos ingresan por data paths diferentes.

Modelo o Arquitectura Harvard

- Esta reliquia ni electrónica ni binaria tenía en sus entrañas la solución al “Von Newmann bottleneck”. Así son las cosas. . .
- Su código ingresaba mediante una cinta de papel perforada, y los operandos mediante contadores electromecánicos. Es decir, las instrucciones y los datos ingresan por data paths diferentes.
- Moraleja: ¡Viejos son los trapos!.

Modelo o Arquitectura Harvard

Conclusiones

Modelo o Arquitectura Harvard

Conclusiones

- Un computador tiene organización o Arquitectura Harvard **cuando las instrucciones y los datos se leen desde memorias físicamente diferentes, y por caminos de señal físicamente diferentes.**

Modelo o Arquitectura Harvard

Conclusiones

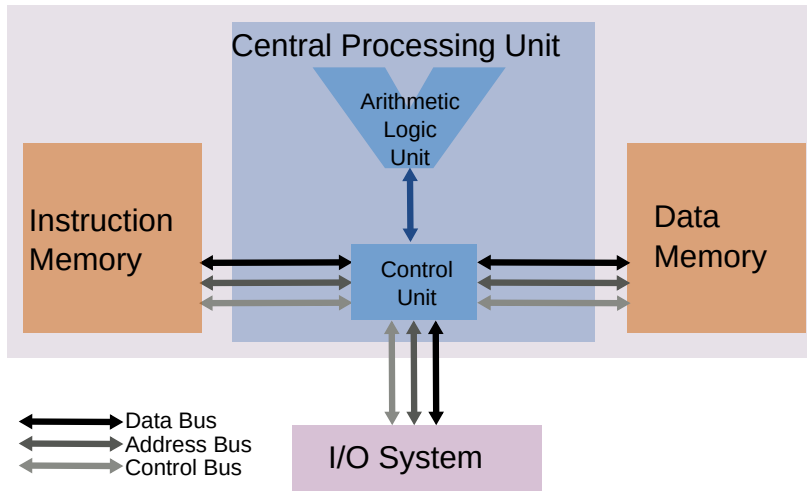
- Un computador tiene organización o Arquitectura Harvard **cuando las instrucciones y los datos se leen desde memorias físicamente diferentes, y por caminos de señal físicamente diferentes.**
- Esto permite tener memorias de tecnología y organización diferente para instrucciones y para datos, y mapas de direcciones particulares en cada caso, y **caminos de señal (buses) independientes.**

Modelo o Arquitectura Harvard

Conclusiones

- Un computador tiene organización o Arquitectura Harvard **cuando las instrucciones y los datos se leen desde memorias físicamente diferentes, y por caminos de señal físicamente diferentes.**
- Esto permite tener memorias de tecnología y organización diferente para instrucciones y para datos, y mapas de direcciones particulares en cada caso, y **caminos de señal (buses) independientes.**
- O sea podemos tener dos direcciones 0x00000000 de memoria: una para instrucciones y otra para datos. La memoria de instrucciones podría estar organizada en palabras de 32 bits y la de datos en palabras de 8 bits. Y todo funciona.

Modelo o Arquitectura Harvard



Arquitectura Harvard. Mapas y buses de memoria diferentes para Instrucciones y Datos

Modelo o Arquitectura Harvard

- La inmensa mayoría de los microprocesadores actuales parecieran presentar al usuario una organización acorde al modelo de Von Newman.

Modelo o Arquitectura Harvard

- La inmensa mayoría de los microprocesadores actuales parecieran presentar al usuario una organización acorde al modelo de Von Newman.
- Sin embargo como veremos en pocas clase mas, en su interior por razones de performance utilizan desde la década del 90 una tecnología denominada split cache: dividir los primeros niveles de memoria cache en una de Instrucciones y otra de datos, con sendos buses independientes para accederlas en paralelo. Esto es Harvard.

Modelo o Arquitectura Harvard

- La inmensa mayoría de los microprocesadores actuales parecieran presentar al usuario una organización acorde al modelo de Von Newman.
- Sin embargo como veremos en pocas clase mas, en su interior por razones de performance utilizan desde la década del 90 una tecnología denominada split cache: dividir los primeros niveles de memoria cache en una de Instrucciones y otra de datos, con sendos buses independientes para accederlas en paralelo. Esto es Harvard.
- Sin embargo, los niveles de memoria mas externos almacenan instrucciones y datos juntos. Esto es Von Newmann.

Modelo o Arquitectura Harvard

- El sub-sistema de memoria cache está diseñado y dimensionado de modo que la mayoría de los accesos a memoria de parte de las CPUs se producen en el cache, es decir, en la memoria organizada de acuerdo al modelo de Harvard.

Modelo o Arquitectura Harvard

- El sub-sistema de memoria cache está diseñado y dimensionado de modo que la mayoría de los accesos a memoria de parte de las CPUs se producen en el cache, es decir, en la memoria organizada de acuerdo al modelo de Harvard.
- La única salvedad por la que no termina siendo “estrictamente Harvard” ni siquiera el primer nivel de cache está dado por el hecho que el programador “ve” una sola dirección de memoria en lugar de ver una de datos y otra de código. Esta división la realiza la CPU (en las MMU por ejemplo según veremos).

Modelo o Arquitectura Harvard

- Los primeros modelos de microprocesadores basados en arquitectura Harvard se dieron en la década del 80 con el advenimiento de los DSP (Procesadores de Señales Digitales), que requerían un alto nivel de paralelismo.

Modelo o Arquitectura Harvard

- Los primeros modelos de microprocesadores basados en arquitectura Harvard se dieron en la década del 80 con el advenimiento de los DSP (Procesadores de Señales Digitales), que requerían un alto nivel de paralelismo.
- Estos a la fecha son los únicos procesadores puramente Harvard.

Modelo o Arquitectura Harvard

- Los primeros modelos de microprocesadores basados en arquitectura Harvard se dieron en la década del 80 con el advenimiento de los DSP (Procesadores de Señales Digitales), que requerían un alto nivel de paralelismo.
- Estos a la fecha son los únicos procesadores puramente Harvard.
- Los procesadores de propósito general como Intel x86, ARM CORTEX-A, CORTEX-R, y últimamente algún que otro CORTEX-M, Power, entre otros trabajan con split cache en el Nivel 1 de su jerarquía de memoria, y juntan código y datos en los niveles mas externos de memoria.

Modelo o Arquitectura Harvard

- Los primeros modelos de microprocesadores basados en arquitectura Harvard se dieron en la década del 80 con el advenimiento de los DSP (Procesadores de Señales Digitales), que requerían un alto nivel de paralelismo.
- Estos a la fecha son los únicos procesadores puramente Harvard.
- Los procesadores de propósito general como Intel x86, ARM CORTEX-A, CORTEX-R, y últimamente algún que otro CORTEX-M, Power, entre otros trabajan con split cache en el Nivel 1 de su jerarquía de memoria, y juntan código y datos en los niveles mas externos de memoria.
- Todos presentan al programador un mapa de direcciones de memoria único. Razón por la cual se los llama “casi - Von Neumann”.

- 1 Introducción
- 2 Objetivos de estudiar organización y hardware
- 3 Orígenes**
 - En solo 80 años
 - **Modernidad**
 - Pipeline
- 4 Nace un paradigma de Arquitectura: RISC

2da. Generación

2da. Generación

- Son los computadores transistorizados.

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell. (patente en Canadá en 1925)

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell. (patente en Canadá en 1925)
- Los primeros equipos aparecen durante fines de la década del 50.

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell. (patente en Canadá en 1925)
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell. (patente en Canadá en 1925)
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.
- Coincide con los lenguajes de Alto Nivel. En 1957 se escribe el primer compilador de FORTRAN (**FOR**mula **TRAN**slator).

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell. (patente en Canadá en 1925)
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.
- Coincide con los lenguajes de Alto Nivel. En 1957 se escribe el primer compilador de FORTRAN (**FOR**mula **TRAN**slator).
- Bell Labs presenta en 1958 la TRADIC (**TR**Ansistor, **D**igital **C**omputer)

2da. Generación

- Son los computadores transistorizados.
- El transistor se implementa en 1947 en Laboratorios Bell. (patente en Canadá en 1925)
- Los primeros equipos aparecen durante fines de la década del 50.
- Ventajas: Menor consumo, menor espacio, y (a largo plazo) mejor desempeño.
- Coincide con los lenguajes de Alto Nivel. En 1957 se escribe el primer compilador de FORTRAN (**FOR**mula **TRAN**slator).
- Bell Labs presenta en 1958 la TRADIC (**TR**Ansistor, **D**igital **C**omputer)
- Se consolida IBM, aparece XEROX y DEC.

3er. Generación

3er. Generación

- Computadores basados en circuitos integrados. Jack Kirby (TI) lo patenta junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo)

3er. Generación

- Computadores basados en circuitos integrados. Jack Kirby (TI) lo patenta junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo)
- En 1957 Robert Noyce funda Fairchild, y luego del patentamiento de Kirby le agrega una capa metálica que permite mejorar su conexionado.

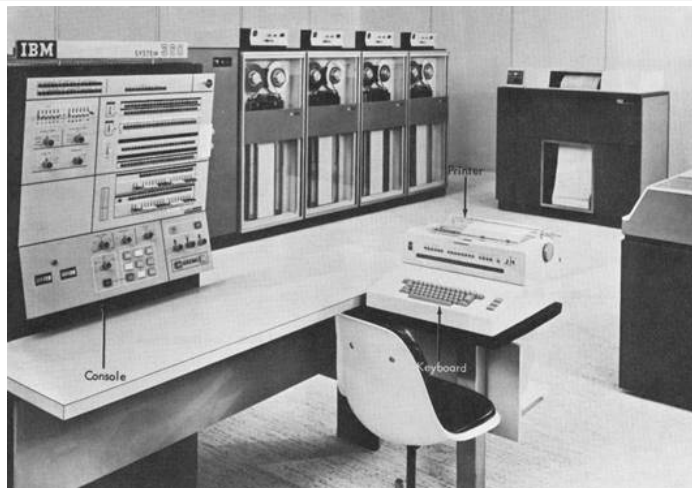
3er. Generación

- Computadores basados en circuitos integrados. Jack Kirby (TI) lo patenta junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo)
- En 1957 Robert Noyce funda Fairchild, y luego del patentamiento de Kirby le agrega una capa metálica que permite mejorar su conexonado.
- Igual que para la segunda generación, transcurriría un tiempo desde el descubrimiento del CI y la implementación de un computador de esta nueva generación.

3er. Generación

- Computadores basados en circuitos integrados. Jack Kirby (TI) lo patenta junto con una implementación en 1959. (Jacobi, Siemens AG había patentado también en 1958 solo el dispositivo)
- En 1957 Robert Noyce funda Fairchild, y luego del patentamiento de Kirby le agrega una capa metálica que permite mejorar su conexonado.
- Igual que para la segunda generación, transcurriría un tiempo desde el descubrimiento del CI y la implementación de un computador de esta nueva generación.
- Los primeros equipos ven la luz en los años 60. El paisaje de un salón de cómputos comienza a verse muy diferente.

IBM 360. Minicomputador de 1964.



Cabe en una mesa. Los demás módulos son periféricos.

PDP-8. Minicomputador de 1964



Por su parte DEC lanza en el mismo año los PDP-8 que también podían ubicarse sobre una mesa pequeña.

Se consolida la Tecnología CMOS

Se consolida la Tecnología CMOS

- En los años 60 Los transistores de Efecto de Campo ya son una realidad y los MOS-FET comienzan lentamente a impulsar la industria de los semiconductores.

Se consolida la Tecnología CMOS

- En los años 60 Los transistores de Efecto de Campo ya son una realidad y los MOS-FET comienzan lentamente a impulsar la industria de los semiconductores.
- La miniaturización de los componentes de un circuito integrado avanza rápidamente, de modo que el tamaño de computadores de esta generación es significativamente menor.

Se consolida la Tecnología CMOS

- En los años 60 Los transistores de Efecto de Campo ya son una realidad y los MOS-FET comienzan lentamente a impulsar la industria de los semiconductores.
- La miniaturización de los componentes de un circuito integrado avanza rápidamente, de modo que el tamaño de computadores de esta generación es significativamente menor.
- En 1968 Robert Noyce deja Fairchild, y funda Intel junto a Gordon Moore y Andrew Grove. Fue su primer CEO.

Se consolida la Tecnología CMOS

- En que se transformó Intel y su contribución a la industria y la ciencia de computación, junto con Texas, Fairchild, y otras empresas, es historia conocida.

Se consolida la Tecnología CMOS

- En que se transformó Intel y su contribución a la industria y la ciencia de computación, junto con Texas, Fairchild, y otras empresas, es historia conocida.
- Esto genera un desarrollo impresionante en el Valle de Santa Clara en California del Norte, zona que pasa a conocerse como Silicon Valley.

Se consolida la Tecnología CMOS

- En que se transformó Intel y su contribución a la industria y la ciencia de computación, junto con Texas, Fairchild, y otras empresas, es historia conocida.
- Esto genera un desarrollo impresionante en el Valle de Santa Clara en California del Norte, zona que pasa a conocerse como Silicon Valley.
- La figura de Robert Noyce fue tan crucial en este proceso que se lo suele referir como el Alcalde del Silicon Valley.

Se consolida la Tecnología CMOS

- En que se transformó Intel y su contribución a la industria y la ciencia de computación, junto con Texas, Fairchild, y otras empresas, es historia conocida.
- Esto genera un desarrollo impresionante en el Valle de Santa Clara en California del Norte, zona que pasa a conocerse como Silicon Valley.
- La figura de Robert Noyce fue tan crucial en este proceso que se lo suele referir como el Alcalde del Silicon Valley.
- Ingresamos a una vorágine tecnológica que transformará la forma de vida de los próximos 40 años de manera sustancial.

3er. Generación = Minicomputadores

Minicomputadores

Costaban un orden de cantidad menos que las enormes computadores transistorizados. Sus fabricantes podían fabricar en serie algunos miles de equipos y así se puede empezar a desarrollar la industria.

3er. Generación = Programación estructurada

Programación Estructurada y Lenguaje C

3er. Generación = Programación estructurada

Programación Estructurada y Lenguaje C

- Se impone el paradigma de programación estructurada durante los años 60.

3er. Generación = Programación estructurada

Programación Estructurada y Lenguaje C

- Se impone el paradigma de programación estructurada durante los años 60.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en una PDP-7.

3er. Generación = Programación estructurada

Programación Estructurada y Lenguaje C

- Se impone el paradigma de programación estructurada durante los años 60.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en una PDP-7.
- A principios de los años 70 Ritchie libera el primer compilador de lenguaje C, documenta la sintaxis del lenguaje,

3er. Generación = Programación estructurada

Programación Estructurada y Lenguaje C

- Se impone el paradigma de programación estructurada durante los años 60.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en una PDP-7.
- A principios de los años 70 Ritchie libera el primer compilador de lenguaje C, documenta la sintaxis del lenguaje,
- En 1970 Ritchie y Thompson Thompson escriben la primer versión de **UNIX** completamente portable cuyo código está escrito prácticamente en su totalidad en C.

3er. Generación = Programación estructurada

Programación Estructurada y Lenguaje C

- Se impone el paradigma de programación estructurada durante los años 60.
- En 1969 Dennis Ritchie y Ken Thompson presentan en Bell Labs un sistema operativo multitarea y multiusuario llamado UNIX, en una PDP-7.
- A principios de los años 70 Ritchie libera el primer compilador de lenguaje C, documenta la sintaxis del lenguaje,
- En 1970 Ritchie y Thompson escriben la primer versión de **UNIX** completamente portable cuyo código está escrito prácticamente en su totalidad en C.
- El C se impone para escribir sistemas operativos, introduce el concepto de portabilidad y también para desarrollar aplicaciones escalables y portables.

Aparecen los primeros microprocesadores

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.
- Desarrollado a pedido para Busicom una compañía Japonesa fabricante de Calculadoras.

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.
- Desarrollado a pedido para Busicom una compañía Japonesa fabricante de Calculadoras.
- Arquitectura de 4 bits Turing Completa (16 registros de 4 bits).

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.
- Desarrollado a pedido para Busicom una compañía Japonesa fabricante de Calculadoras.
- Arquitectura de 4 bits Turing Completa (16 registros de 4 bits).
- Noyce contrata a Federico Faggin de Fairchild autor de la Silicon Gate Technology (SGT) tecnología crucial para obtener la integración de los 2300 transistores que componían el 4004.

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.
- Desarrollado a pedido para Busicom una compañía Japonesa fabricante de Calculadoras.
- Arquitectura de 4 bits Turing Completa (16 registros de 4 bits).
- Noyce contrata a Federico Faggin de Fairchild autor de la Silicon Gate Technology (SGT) tecnología crucial para obtener la integración de los 2300 transistores que componían el 4004.
- Tecnología de 15 μm (o 15 micras según la jerga de esa época), clock de 740KHz., y 12 bits para direcciones (4KB de memoria).

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.
- Desarrollado a pedido para Busicom una compañía Japonesa fabricante de Calculadoras.
- Arquitectura de 4 bits Turing Completa (16 registros de 4 bits).
- Noyce contrata a Federico Faggin de Fairchild autor de la Silicon Gate Technology (SGT) tecnología crucial para obtener la integración de los 2300 transistores que componían el 4004.
- Tecnología de $15\ \mu m$ (o 15 micras según la jerga de esa época), clock de 740KHz., y 12 bits para direcciones (4KB de memoria).
- Espacios de direcciones separados para instrucciones y datos

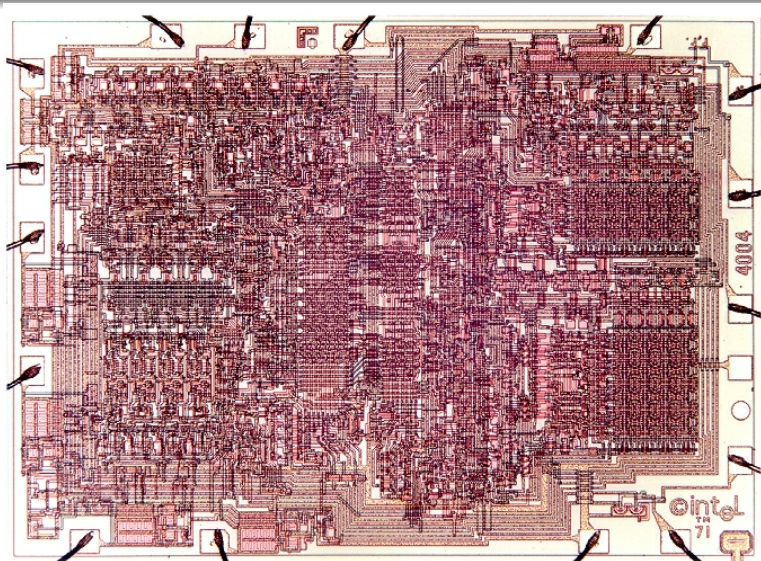
Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.
- Desarrollado a pedido para Busicom una compañía Japonesa fabricante de Calculadoras.
- Arquitectura de 4 bits Turing Completa (16 registros de 4 bits).
- Noyce contrata a Federico Faggin de Fairchild autor de la Silicon Gate Technology (SGT) tecnología crucial para obtener la integración de los 2300 transistores que componían el 4004.
- Tecnología de 15 μm (o 15 micras según la jerga de esa época), clock de 740KHz., y 12 bits para direcciones (4KB de memoria).
- Espacios de direcciones separados para instrucciones y datos
- Se le atribuye en forma errónea en ciertos documentos arquitectura Harvard

Aparecen los primeros microprocesadores

- Intel fabricaba CI de memorias.
- En 1971 presenta el CI 4004. Primer Microprocesador de la historia.
- Desarrollado a pedido para Busicom una compañía Japonesa fabricante de Calculadoras.
- Arquitectura de 4 bits Turing Completa (16 registros de 4 bits).
- Noyce contrata a Federico Faggin de Fairchild autor de la Silicon Gate Technology (SGT) tecnología crucial para obtener la integración de los 2300 transistores que componían el 4004.
- Tecnología de $15\ \mu m$ (o 15 micras según la jerga de esa época), clock de 740KHz., y 12 bits para direcciones (4KB de memoria).
- Espacios de direcciones separados para instrucciones y datos
- Se le atribuye en forma errónea en ciertos documentos arquitectura Harvard
- 16 terminales. Multiplexa la dirección de 12 bits con los mismos 4 bits de datos.

El 4004 de Intel



Se empieza a acelerar la industria

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, consolidándose como fabricante de Microprocesadores y memorias.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, consolidándose como fabricante de Microprocesadores y memorias.
- En 1974 Faggin funda ZILOG y lanza el Z80, que supera claramente al 8080 y al 8085 con el que Intel intentará contrarrestarlo.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, consolidándose como fabricante de Microprocesadores y memorias.
- En 1974 Faggin funda ZILOG y lanza el Z80, que supera claramente al 8080 y al 8085 con el que Intel intentará contrarrestarlo.
- Si IBM no hubiese seleccionado un proyecto de procesador de 16 bits en curso de Intel (el 8086) como procesador de su primer computadora personal otra sería la historia tal vez.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, consolidándose como fabricante de Microprocesadores y memorias.
- En 1974 Faggin funda ZILOG y lanza el Z80, que supera claramente al 8080 y al 8085 con el que Intel intentará contrarrestarlo.
- Si IBM no hubiese seleccionado un proyecto de procesador de 16 bits en curso de Intel (el 8086) como procesador de su primer computadora personal otra sería la historia tal vez.
- Primeros computadores personales. Altair, IBM PC, Apple, Commodore, Sinclair, MSX, etc. utilizaron generaciones de procesadores de 8 o 16 bits.

Se empieza a acelerar la industria

- Con la tecnología SGT Intel lanza los procesadores 8008, 4040 y 8080, consolidándose como fabricante de Microprocesadores y memorias.
- En 1974 Faggin funda ZILOG y lanza el Z80, que supera claramente al 8080 y al 8085 con el que Intel intentará contrarrestarlo.
- Si IBM no hubiese seleccionado un proyecto de procesador de 16 bits en curso de Intel (el 8086) como procesador de su primer computadora personal otra sería la historia tal vez.
- Primeros computadores personales. Altair, IBM PC, Apple, Commodore, Sinclair, MSX, etc. utilizaron generaciones de procesadores de 8 o 16 bits.
- Hasta mitad de los 80 estamos en esta etapa. Aparecen Microsoft y Apple como actores nuevos de esta historia. El DOS se impone como Sistema Operativo y dominaría la década del 80. Aun entrada la cuarta generación.

Y UNIX ya había llegado en 1969



Esta máquina es a la que se portó la primer versión escrita en C.

16 Kbytes de memoria RAM.

512 Kbytes de almacenamiento...

4ta. Generación

4ta. Generación

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore.

4ta. Generación

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore.
- Al momento de escribir este texto, la Ley de Moore está llegando a su fin, sencillamente porque se han alcanzado dimensiones físicas de niveles atómicos, de modo que ya no es posible reducir mucho más las dimensiones de una compuerta de un transistor MOSFET.

4ta. Generación

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore.
- Al momento de escribir este texto, la Ley de Moore está llegando a su fin, sencillamente porque se han alcanzado dimensiones físicas de niveles atómicos, de modo que ya no es posible reducir mucho mas las dimensiones de una compuerta de un transistor MOSFET.
- Alrededor de 1980 se comienza a hablar de Circuitos Integrados VLSI (Very Large Scale Integration), en referencia a circuitos integrados que contienen cientos de miles de transistores integrados.

4ta. Generación

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore.
- Al momento de escribir este texto, la Ley de Moore está llegando a su fin, sencillamente porque se han alcanzado dimensiones físicas de niveles atómicos, de modo que ya no es posible reducir mucho más las dimensiones de una compuerta de un transistor MOSFET.
- Alrededor de 1980 se comienza a hablar de Circuitos Integrados VLSI (Very Large Scale Integration), en referencia a circuitos integrados que contienen cientos de miles de transistores integrados.
- La verdad es que de lo que se ha producido en esta era es de lo que vamos a hablar en el resto de ésta asignatura y la siguiente.

4ta. Generación

- El desarrollo de la tecnología de integración evolucionó de acuerdo con la predicción de Gordon Moore.
- Al momento de escribir este texto, la Ley de Moore está llegando a su fin, sencillamente porque se han alcanzado dimensiones físicas de niveles atómicos, de modo que ya no es posible reducir mucho mas las dimensiones de una compuerta de un transistor MOSFET.
- Alrededor de 1980 se comienza a hablar de Circuitos Integrados VLSI (Very Large Scale Integration), en referencia a circuitos integrados que contienen cientos de miles de transistores integrados.
- La verdad es que de lo que se ha producido en esta era es de lo que vamos a hablar en el resto de ésta asignatura y la siguiente.
- Lo divertido es que estamos transitándola aún

¿Porqué esta clase de historia?

¿Porqué esta clase de historia?

- Actualmente la preocupación central de los fabricantes de microprocesadores es el consumo de energía eléctrica. todos investigan y trabajan diariamente para presentar chips que consuman el mínimo de energía para el mismo trabajo.

¿Porqué esta clase de historia?

- Actualmente la preocupación central de los fabricantes de microprocesadores es el consumo de energía eléctrica. todos investigan y trabajan diariamente para presentar chips que consuman el mínimo de energía para el mismo trabajo.
- Durante el corto lapso que lleva esta industria, la memoria fue durante muchas décadas el recurso mas escaso (y en rigor sigue vigente la máxima de Von Neumann “*la cantidad de memoria siempre será insuficiente*”). Pero como pudimos apreciar, en los albores esta escasez fue dramática.

¿Porqué esta clase de historia?

- Actualmente la preocupación central de los fabricantes de microprocesadores es el consumo de energía eléctrica. todos investigan y trabajan diariamente para presentar chips que consuman el mínimo de energía para el mismo trabajo.
- Durante el corto lapso que lleva esta industria, la memoria fue durante muchas décadas el recurso mas escaso (y en rigor sigue vigente la máxima de Von Neumann “*la cantidad de memoria siempre será insuficiente*”). Pero como pudimos apreciar, en los albores esta escasez fue dramática.
- 16 Kbytes de RAM (8 para el sistema operativo y 8 para las aplicaciones) en 1969,... hue las palabras.

Uso mínimo de memoria: un “must”

Por lo tanto así como hoy se diseñan procesadores con foco puesto en el mínimo consumo de energía, hace 4 o 5 décadas, se diseñaban pensando en que los programas consuman la mínima cantidad de memoria posible.

Instrucciones complejas

- 1 Los diseñadores de la época apuntaban a microprocesadores con instrucciones muy complejas, para lograr códigos muy compactos (y que ocupen poca memoria!)

Instrucciones complejas

- 1 Los diseñadores de la época apuntaban a microprocesadores con instrucciones muy complejas, para lograr códigos muy compactos (y que ocupen poca memoria!)
- 2 En ese contexto se han diseñado Microprocesadores capaces de integrar código de alto nivel.

Instrucciones complejas

- 1 Los diseñadores de la época apuntaban a microprocesadores con instrucciones muy complejas, para lograr códigos muy compactos (y que ocupen poca memoria!)
- 2 En ese contexto se han diseñado Microprocesadores capaces de integrar código de alto nivel.
- 3 Ejemplos extremos: POLY (VAX computer)

Instrucciones complejas

- 1 Los diseñadores de la época apuntaban a microprocesadores con instrucciones muy complejas, para lograr códigos muy compactos (y que ocupen poca memoria!)
- 2 En ese contexto se han diseñado Microprocesadores capaces de integrar código de alto nivel.
- 3 Ejemplos extremos: POLY (VAX computer)

Listado1

```
; To compute  $P(x) = C0 + C1*x + C2*x**2$ 
; where  $C0 = 1.0$ ,  $C1 = .5$ , and
 $C2 = .25$ 
```

```
        POLYF      X,#2,PTABLE
        .
        .
        .
PTABLE: .FLOAT      0.25      ; C2
        .FLOAT      0.5       ; C1
        .FLOAT      1.0       ; C0
```


Instrucciones complejas: Conclusión

Con los bloques de datos no hay mucho que hacer para ahorrar memoria. Pero en el caso del código, si podemos utilizar menos instrucciones para el mismo trabajo, se ahorra memoria. Este paradigma estuvo presente en todos los diseños durante décadas. Intel desarrolló su arquitectura actual con este paradigma. Y se comprometió a mantener **compatibilidad**.

Otros problemas a resolver

- En los '70, los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos

Otros problemas a resolver

- En los '70, los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos
- Cada upgrade obligaba a reemplazar no solo el hardware sino además el sistema operativo y las aplicaciones.

Otros problemas a resolver

- En los '70, los usuarios de computadores demandaban disminuir el costo de actualización de sus equipos
- Cada upgrade obligaba a reemplazar no solo el hardware sino además el sistema operativo y las aplicaciones.
- Ésta dinámica dificultaba a los proveedores establecer un cash flow que permitiese incrementar inversiones en I+D.

Intel dá el golpe

Intel dá el golpe

- En 1978 presenta la familia iAPx86 de la mano de su nave insignia procesador 8086,

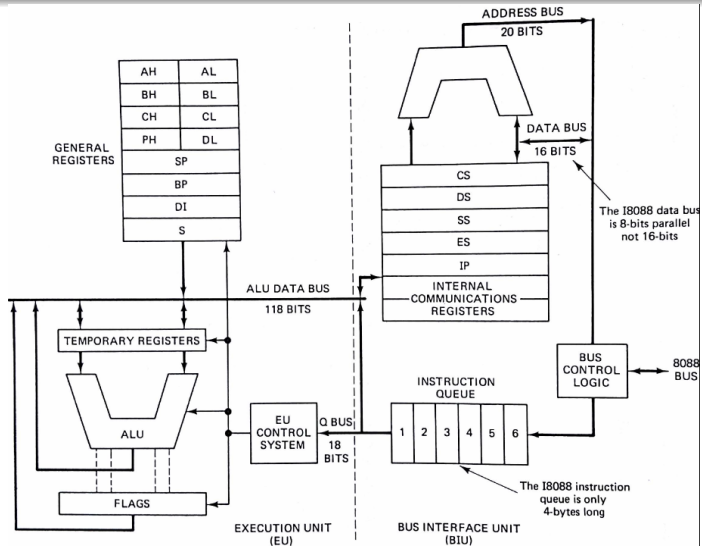
Intel dá el golpe

- En 1978 presenta la familia iAPx86 de la mano de su nave insignia procesador 8086,
- Se compromete a mantener compatibilidad ascendente en los modelos subsiguientes de procesadores para satisfacer esta demanda.

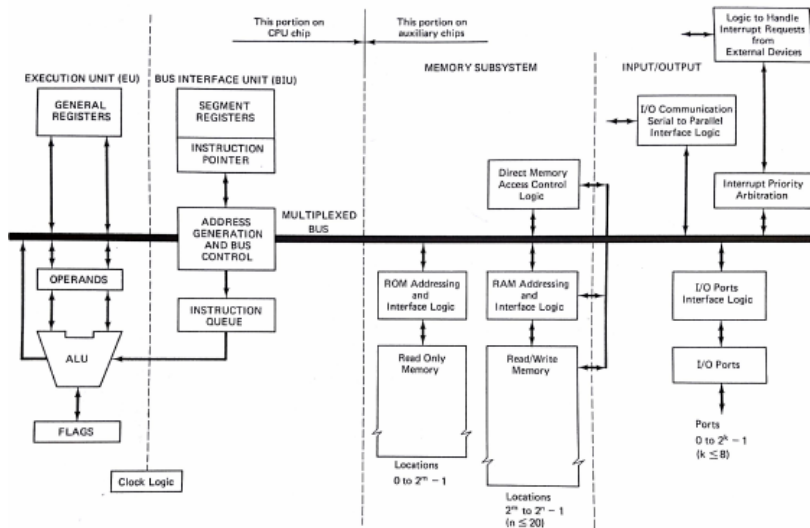
Intel dá el golpe

- En 1978 presenta la familia iAPx86 de la mano de su nave insignia procesador 8086,
- Se compromete a mantener compatibilidad ascendente en los modelos subsiguientes de procesadores para satisfacer esta demanda.
- Esto fue uno de los motivos de la decisión de IBM de adoptar esta familia de procesadores como base para su PC.

8086 Organización



Sistema básico de 8086



- 1 Introducción
- 2 Objetivos de estudiar organización y hardware
- 3 Orígenes**
 - En solo 80 años
 - Modernidad
 - **Pipeline**
- 4 Nace un paradigma de Arquitectura: RISC

Máquina de estados elemental

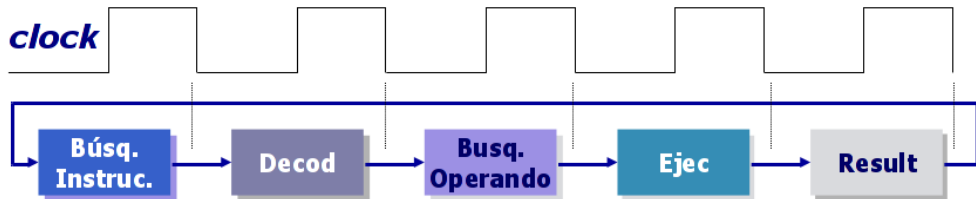
- En la década del 40 Von Newman definió un modelo básico de CPU, que a esta altura está mas que superado.

Máquina de estados elemental

- En la década del 40 Von Newman definió un modelo básico de CPU, que a esta altura está mas que superado.
- Sin embargo algunos conceptos de ese modelo viven en los mas modernos procesadores.

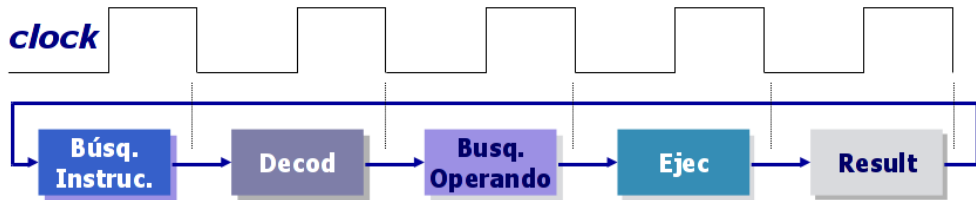
Máquina de estados elemental

- En la década del 40 Von Newman definió un modelo básico de CPU, que a esta altura está mas que superado.
- Sin embargo algunos conceptos de ese modelo viven en los mas modernos procesadores.
- Uno de ellos es la máquina de ejecución



Máquina de estados elemental

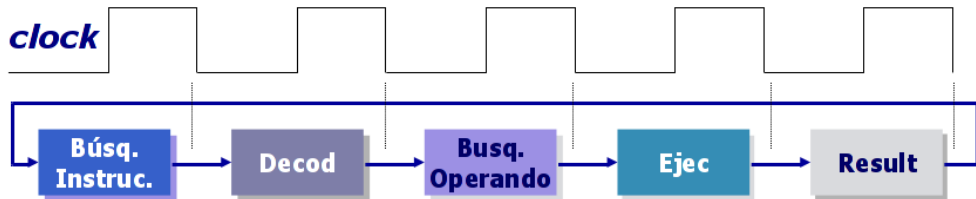
- En la década del 40 Von Newman definió un modelo básico de CPU, que a esta altura está mas que superado.
- Sin embargo algunos conceptos de ese modelo viven en los mas modernos procesadores.
- Uno de ellos es la máquina de ejecución



- En las primeras generaciones de microprocesadores cada etapa de este ciclo se ejecutaba en un ciclo de clock, y la CPU entera estaba dedicada a esa tarea.

Máquina de estados elemental

- En la década del 40 Von Newman definió un modelo básico de CPU, que a esta altura está mas que superado.
- Sin embargo algunos conceptos de ese modelo viven en los mas modernos procesadores.
- Uno de ellos es la máquina de ejecución



- En las primeras generaciones de microprocesadores cada etapa de este ciclo se ejecutaba en un ciclo de clock, y la CPU entera estaba dedicada a esa tarea.
- Ejecutar una instrucción insumía de varios ciclos de clock...

Pipeline

- Arquitectura que permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.

Pipeline

- Arquitectura que permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Con él se formaliza el concepto de *Instruction Level Paralelism (ILP)*.

Pipeline

- Arquitectura que permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Con él se formaliza el concepto de *Instruction Level Parallelism (ILP)*.
- Requiere muy poco o ningún hardware adicional.

Pipeline

- Arquitectura que permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Con él se formaliza el concepto de *Instruction Level Parallelism (ILP)*.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados para la ejecución de una instrucción, operen en forma simultánea.

Pipeline

- Arquitectura que permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Con él se formaliza el concepto de **Instruction Level Parallelism (ILP)**.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados para la ejecución de una instrucción, operen en forma simultánea.
- Se logra si todos los bloques funcionales trabajan en paralelo pero cada uno en una instrucción diferente.

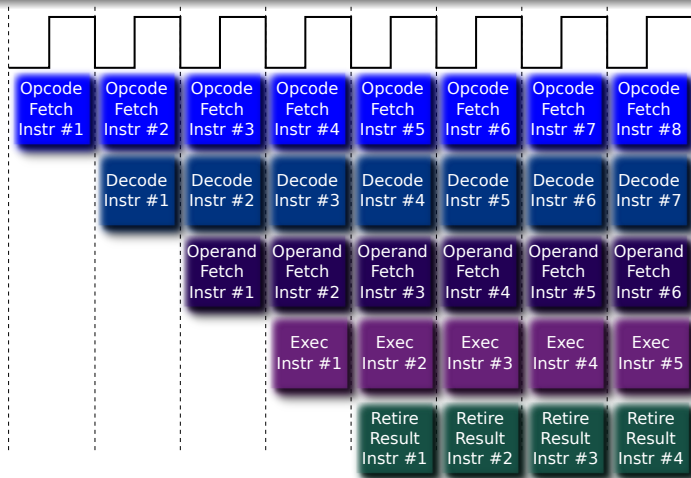
Pipeline

- Arquitectura que permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Con él se formaliza el concepto de *Instruction Level Parallelism (ILP)*.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados para la ejecución de una instrucción, operen en forma simultánea.
- Se logra si todos los bloques funcionales trabajan en paralelo pero cada uno en una instrucción diferente.
- Es algo parecido al concepto de una línea de montaje, en donde cada operación se descompone en partes, y se ejecutan en un mismo momento diferentes partes de diferentes operaciones.

Pipeline

- Arquitectura que permite crear el efecto de superponer en el tiempo, la ejecución de varias instrucciones a la vez.
- Con él se formaliza el concepto de **Instruction Level Parallelism (ILP)**.
- Requiere muy poco o ningún hardware adicional.
- Solo necesita que los bloques del procesador que resuelven la máquina de estados para la ejecución de una instrucción, operen en forma simultánea.
- Se logra si todos los bloques funcionales trabajan en paralelo pero cada uno en una instrucción diferente.
- Es algo parecido al concepto de una línea de montaje, en donde cada operación se descompone en partes, y se ejecutan en un mismo momento diferentes partes de diferentes operaciones.
- Cada parte se denomina etapa (stage)

Pipeline



Este modelo es teórico y representa la situación ideal.

Obstáculos Estructurales: Ejemplo

- Tenemos un procesador que solo tiene una etapa para acceder a memoria y la comparte para acceso a datos e instrucciones.

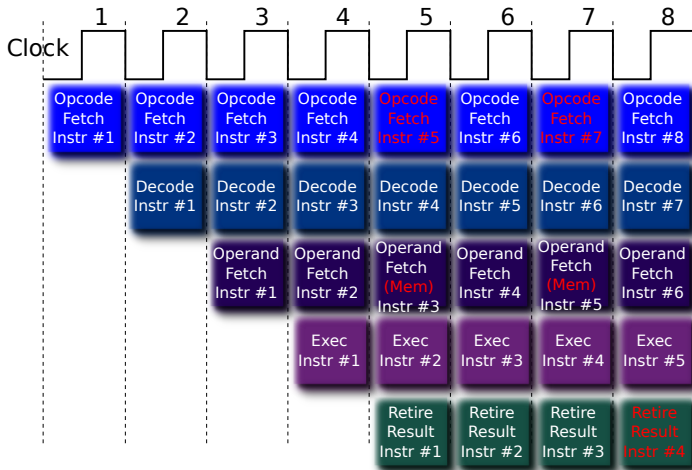
Obstáculos Estructurales: Ejemplo

- Tenemos un procesador que solo tiene una etapa para acceder a memoria y la comparte para acceso a datos e instrucciones.
- En el caso de que se necesite un operando de memoria, el acceso para traer este operando interferirá con la búsqueda del operando de una instrucción mas adelante del programa.

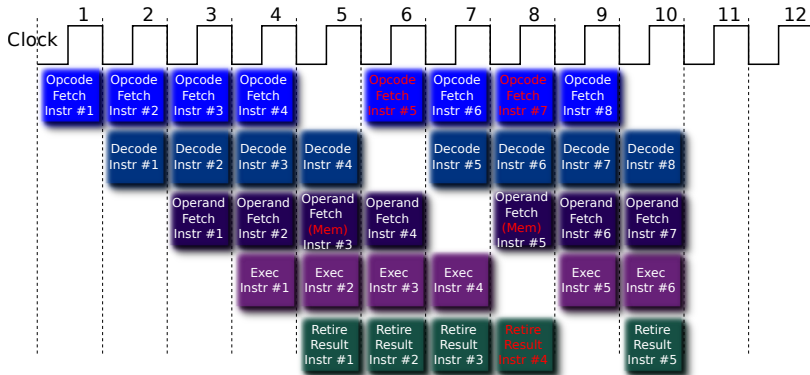
Obstáculos Estructurales: Ejemplo

- Tenemos un procesador que solo tiene una etapa para acceder a memoria y la comparte para acceso a datos e instrucciones.
- En el caso de que se necesite un operando de memoria, el acceso para traer este operando interferirá con la búsqueda del operando de una instrucción mas adelante del programa.
- También interferirá con el Fetch de la siguiente instrucción

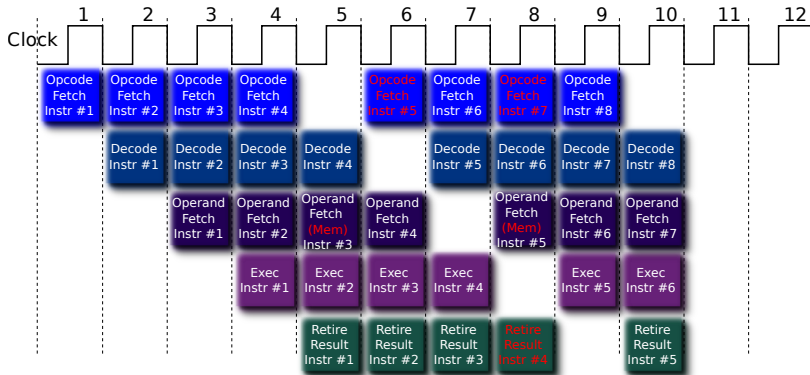
Obstáculos Estructurales: Accesos concurrentes a memoria



Obstáculos Estructurales - Efecto en CPI

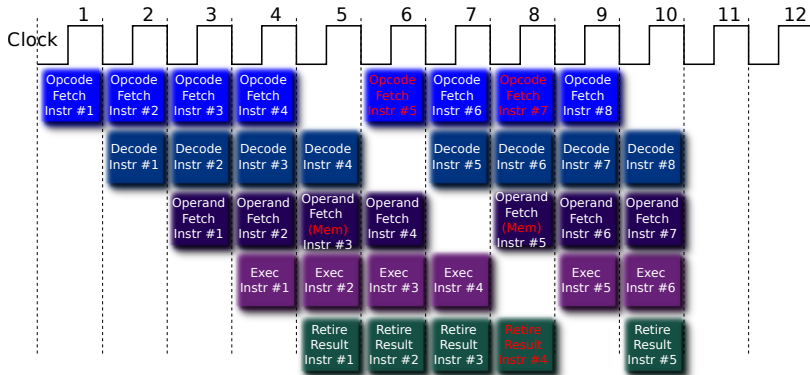


Obstáculos Estructurales - Efecto en CPI



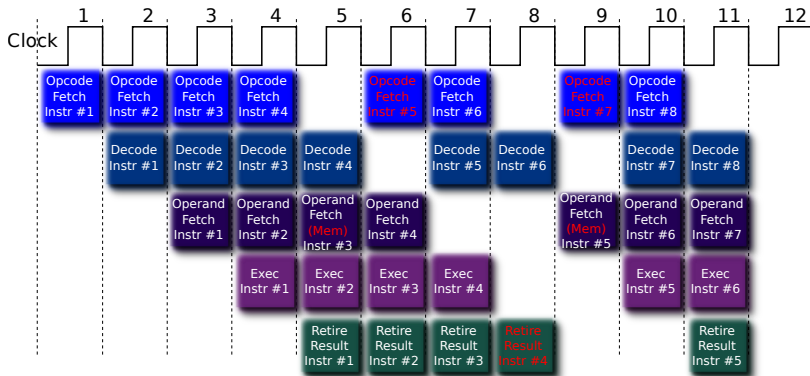
- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$

Obstáculos Estructurales - Efecto en CPI



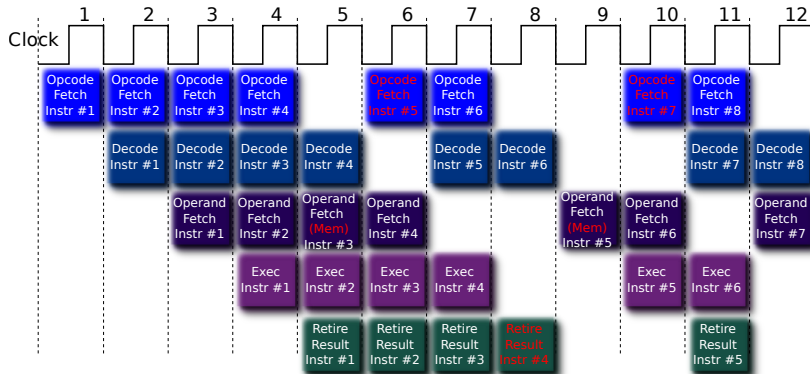
- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$
- En general la cantidad de CPI que se incrementan es igual a la cantidad de concurrencias menos 1, en el lapso considerado

Obstáculos Estructurales - Efecto en CPI



- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$
- En general la cantidad de CPI que se incrementan es igual a la cantidad de concurrencias menos 1, en el lapso considerado

Obstáculos Estructurales - Efecto en CPI



- Por cada Obstáculo, pospone una operación. $CPI = CPI + 1$
- En general la cantidad de CPI que se incrementan es igual a la cantidad de concurrencias menos 1, en el lapso considerado

Obstáculos de Control

Obstáculos de Control

- Un branch es la peor situación en pérdida de performance.

Obstáculos de Control

- Un branch es la peor situación en pérdida de performance.
- Un branch es una discontinuidad en el flujo de ejecución.

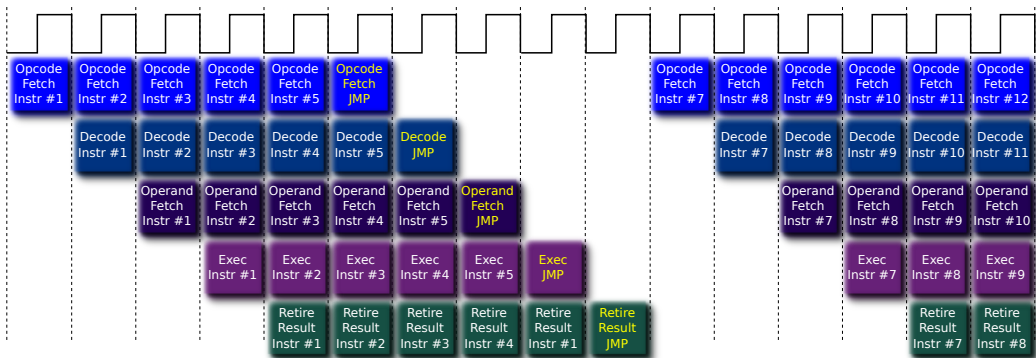
Obstáculos de Control

- Un branch es la peor situación en pérdida de performance.
- Un branch es una discontinuidad en el flujo de ejecución.
- El pipeline busca instrucciones en secuencia.

Obstáculos de Control

- Un branch es la peor situación en pérdida de performance.
- Un branch es una discontinuidad en el flujo de ejecución.
- El pipeline busca instrucciones en secuencia.
- El branch hace que todo lo que estaba pre procesado deba descartarse. Y el pipeline se vacía debiendo transcurrir $n - 1$ ciclos de clock hasta el próximo resultado. Siendo n la cantidad de etapas del pipeline. Esto se conoce como **branch penalty**.

"La conspiración de los branches"



- 1 Introducción
- 2 Objetivos de estudiar organización y hardware
- 3 Orígenes
- 4 Nace un paradigma de Arquitectura: RISC**
 - **Antecedentes**

Orígenes de RISC

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:
 - Intel evoluciona el 8086/88 al 80286, y ensaya el iAPx432 citado como micromainframe processor, especialmente diseñado para ejecutar lenguajes de alto nivel como Ada.

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:
 - Intel evoluciona el 8086/88 al 80286, y ensaya el iAPx432 citado como micromainframe processor, especialmente diseñado para ejecutar lenguajes de alto nivel como Ada.
 - Motorola evoluciona el 68000 (citado como m68k)

Orígenes de RISC

- Antes de las Personal Computers, el mercado era dominado por los denominados mainframes.
- Los players eran IBM, Digital Electronic Corporation (DEC), Amdahl, entre otros. Sus computadores hoy son piezas de museo.
- Sobre los años 80 aparecen los primeros microprocesadores de 16 bits con ciertas capacidades para hacer frente a una nueva generación de equipos de cómputo:
 - Intel evoluciona el 8086/88 al 80286, y ensaya el iAPx432 citado como micromainframe processor, especialmente diseñado para ejecutar lenguajes de alto nivel como Ada.
 - Motorola evoluciona el 68000 (citado como m68k)
- Vimos que tienen todos un punto en común: Set de instrucciones de complejidad creciente

Historia de RISC

Líneas de Investigación

Durante los años '60 y '70, IBM, Control Data Corporation (CDC), y Data General (DG), incursionaron líneas de investigación tendientes a implementar procesadores con pocas instrucciones simples.

En los '80 David Patterson^{1 2} (Universidad de Berkeley), y John Hennesy³ (Universidad de Stanford) publicaron los primeros trabajos con resultados concretos, que presentaban una arquitectura contrapuesta con la de los Computadores que en ese momento dominaban la industria. la denominaron **RISC** (**R**educed **I**nstruction **S**et **C**omputer). Y esto motivó que a los procesadores diseñados hasta entonces se los etiquetar como **CISC** (Por **C**omplex **I**nstrucion **S**et **C**omputer)

¹ David A. Patterson, Carlo H. Sequin. RISC I: A Reduced Instruction Set VLSI Computer

² David A. Patterson, David R. Ditzel. The case for the reduced instruction set computer

³ Hennesy, J.L., Jouppi, N., Baskett, F., Gill, J. MIPS: A VLSI Processor Architecture. In Proceedings CMU Conference on VLSI Systems and Computations. Computer Science Press, October 1981

Relación con el pipeline

Relación con el pipeline

- Se analizaron los obstáculos en un pipeline.

Relación con el pipeline

- Se analizaron los obstáculos en un pipeline.
- En particular los obstáculos de datos impactan cuando involucran accesos a memoria para buscar operandos.

Relación con el pipeline

- Se analizaron los obstáculos en un pipeline.
- En particular los obstáculos de datos impactan cuando involucran accesos a memoria para buscar operandos.
- Claramente si se evita que los operandos se accedan en memoria, en lugar de cargarlos previamente en un registro tal vez el pipeline pueda mejorarse

Los Mandamientos RISC

Los Mandamientos RISC

- 1 Se dispone de un juego de registros numeroso, todos de propósito general.

Los Mandamientos RISC

- 1 Se dispone de un juego de registros numeroso, todos de propósito general.
- 2 Las instrucciones se ejecutan en un solo ciclo de clock.

Los Mandamientos RISC

- 1 Se dispone de un juego de registros numeroso, todos de propósito general.
- 2 Las instrucciones se ejecutan en un solo ciclo de clock.
- 3 Las instrucciones derivan en códigos de operación de igual formato y tamaño.

Los Mandamientos RISC

- 1 Se dispone de un juego de registros numeroso, todos de propósito general.
- 2 Las instrucciones se ejecutan en un solo ciclo de clock.
- 3 Las instrucciones derivan en códigos de operación de igual formato y tamaño.
- 4 Las instrucciones deben ser sencillas de decodificar. Los números de registros se deben tener la misma ubicación en los códigos de la instrucción y deben requerir la misma cantidad de bits para su decodificación.

Los Mandamientos RISC

- 1 Se dispone de un juego de registros numeroso, todos de propósito general.
- 2 Las instrucciones se ejecutan en un solo ciclo de clock.
- 3 Las instrucciones derivan en códigos de operación de igual formato y tamaño.
- 4 Las instrucciones deben ser sencillas de decodificar. Los números de registros se deben tener la misma ubicación en los códigos de la instrucción y deben requerir la misma cantidad de bits para su decodificación.
- 5 No se utiliza micro código para decodificar instrucciones (no hay instrucciones complejas, como DIV o MUL).

Los Mandamientos RISC

- 1 Se dispone de un juego de registros numeroso, todos de propósito general.
- 2 Las instrucciones se ejecutan en un solo ciclo de clock.
- 3 Las instrucciones derivan en códigos de operación de igual formato y tamaño.
- 4 Las instrucciones deben ser sencillas de decodificar. Los números de registros se deben tener la misma ubicación en los códigos de la instrucción y deben requerir la misma cantidad de bits para su decodificación.
- 5 No se utiliza micro código para decodificar instrucciones (no hay instrucciones complejas, como DIV o MUL).
- 6 Los datos en memoria se acceden mediante instrucciones simples de transferencia: LOAD y STORE.

Corolario

Corolario

- 1 La validación de una máquina RISC es mucho mas simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica. (Recordemos el bug de división del Pentium@90Mhz)

Corolario

- 1 La validación de una máquina RISC es mucho mas simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica. (Recordemos el bug de división del Pentium@90Mhz)
- 2 Es esperable que el tamaño de los programas sea mayor, debido a que lo que los procesadores CISC resuelven con microcódigo, los RISC lo resuelven en el software. Una multiplicación se resuelve siempre por acumulación de sumas.

$$A * B = \sum_{i=1}^A B$$

Corolario

- 1 La validación de una máquina RISC es mucho mas simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica. (Recordemos el bug de división del Pentium@90Mhz)
- 2 Es esperable que el tamaño de los programas sea mayor, debido a que lo que los procesadores CISC resuelven con microcódigo, los RISC lo resuelven en el software. Una multiplicación se resuelve siempre por acumulación de sumas.

$$A * B = \sum_{i=1}^A B$$

- En microcódigo se implementa la instrucción MUL.

Corolario

- 1 La validación de una máquina RISC es mucho mas simple de realizar, y también es menor la probabilidad de liberar al mercado un procesador con defectos de lógica. (Recordemos el bug de división del Pentium@90Mhz)
- 2 Es esperable que el tamaño de los programas sea mayor, debido a que lo que los procesadores CISC resuelven con microcódigo, los RISC lo resuelven en el software. Una multiplicación se resuelve siempre por acumulación de sumas.

$$A * B = \sum_{i=1}^A B$$

- En microcódigo se implementa la instrucción MUL.
- Por software se implementa con una subrutina.

Listado 2

```
; R1 = R1 * R2
otro:  add R1,R1,R1
       sub R2,R2,#1
       bne otro
```


¿Preguntas?